

SWT-CP: Lightweight Trap-Based Free-Rider Mitigation in Federated Learning

Aashnan Rahman

Department of Computer Science and Engineering

Islamic University of Technology

Student ID: 231041006

Abstract—

*Index Terms—*federated learning, free riding, anomaly detection, robust statistics, client auditing, lightweight defense

I. INTRODUCTION

Federated learning (FL) enables multiple clients to train a shared model without transferring their raw data to a central server. In a typical communication round, the server distributes the current global model, participating clients improve it using their local data, and the server aggregates their returned updates, for example through Federated Averaging (FedAvg) [1]. This collaborative arrangement makes learning possible across distributed and privacy-sensitive data sources and has supported large-scale applications [2], [3].

The quality of the resulting global model nevertheless depends on clients actually performing the local work expected of them. In practice, some clients may decide not to contribute meaningful training. A client may have insufficient useful data, limited computation or energy, or may simply wish to conserve its own resources while continuing to receive the model produced by the federation. Such a participant is commonly described as a *free rider* [4], [5]. It obtains the benefit of the collaborative model while contributing little or no genuine learning in return.

Free riding differs from an active poisoning attack. A free rider does not necessarily attempt to insert a harmful objective, corrupt labels, or deliberately steer the global model in an adversarial direction. Its harmful effect is primarily an omission of useful work. Had the client trained honestly, its local data and computation could have produced an informative update and potentially improved the aggregated model. When it does not, the global model must depend more heavily on the remaining contributors. One inactive participant may have little visible effect, but widespread free riding reduces the effective training population, wastes communication and coordination resources, slows or weakens learning, and creates an unfair distribution of computational cost.

Accordingly, this work studies a lightweight server-side method for identifying clients that repeatedly avoid meaningful local training without relying on client-reported performance or resource information.

II. RELATED WORK

A. Free-rider attacks

Lin *et al.* formalized free riding in FL and introduced both fabricated-update attacks and STD-DAGMM detection [4]. Fraboni *et al.* showed that free riders can imitate successive global updates and disguise them with calibrated noise [5]. Zhu *et al.* subsequently constructed a stronger attack that matches geometric properties used by feature-based defenses [6]. Collectively, these works show that replay and arbitrary noise are insufficient evaluation targets: a detector based on a fixed, known update statistic can be imitated by a history-aware attacker.

B. Statistical and Learning-Based Detection

Statistical detectors use update norms, dispersion, cosine similarity, or temporal consistency because these signals are available at the server and relatively inexpensive. Their central weakness is that non-IID honest updates may also be outliers, while adaptive attackers can reproduce the statistics being monitored. **FRAD** improves on a single statistic by combining a DAGMM-style anomaly detector with contribution and reputation models [7]. **PASS** combines parameter auditing with contribution evaluation and considers both resource-poor and strategically selfish free riders [8]. Both enrich the evidence available to the server, but introduce auxiliary models or auditing assumptions, additional calibration, and greater computational cost. Their learned or engineered references may also require adaptation when the model architecture, data heterogeneity, or attacker behavior changes.

C. Challenge-Based and Verifiable Computation

FRIDA seeks evidence of genuine training through privacy-attack techniques [9]. Its membership-inference variants use a shared canary batch, while its property-inference variants test whether inferred client-data properties remain plausible across rounds. This targets the cause of free riding more directly than update geometry, but adds client or server computation, requires individual-update access, and exposes a known canary objective that a capable attacker can deliberately satisfy. Trusted hardware, remote attestation, and cryptographic proof can provide stronger execution guarantees, but require specialized support or substantial proof and communication overhead; they also complicate integration with secure aggregation [10]. The remaining gap is a lightweight server-side method

that creates unpredictable evidence without auxiliary detector training, trusted client reports, public canary data, or specialized hardware.

III. THREAT MODEL AND ATTACKS

There are N attributable clients and a coordinating server. In round t , client i receives $w_{t,i}^{\text{send}}$ and returns $w_{t,i}^{\text{ret}}$. The server derives

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \quad r_{t,i} = \|\Delta_{t,i}\|_2. \quad (1)$$

For an audited client, $w_{t,i}^{\text{send}}$ is a perturbed trap rather than the aggregating global model. Clients receive no trap indicator, and trap-derived updates are excluded from aggregation.

Attackers know only models previously sent to them. They do not know the audit schedule, group membership, anchor identities, thresholds, or ground-truth labels. Four increasingly informed behaviors are evaluated.

FR1—Playback

$$\hat{w}_{t,i} = w_{t-1,i}^{\text{send}}. \quad (2)$$

The client stores the last model it received and submits that model in the next round without training. During its first round, when no previous model exists, it returns the current received model. This is the least expensive attack, but it responds to a new model using stale information and may also create an exact or near-exact send-back.

FR2—Bounded Random Update

$$\hat{w}_{t,i} = w_{t,i}^{\text{send}} + \epsilon, \quad \epsilon_j \sim \mathcal{U}(-10^{-3}, 10^{-3}). \quad (3)$$

FR2 avoids an exact send-back by adding an independently sampled bounded value to every floating-point coordinate. The response is nonzero and superficially update-like, but it contains no information from local data and has no memory of earlier rounds. It represents a client attempting to bypass simple equality or zero-norm checks at negligible computational cost.

FR3—Historical Averaging

$$\hat{w}_{t,i} = \frac{1}{|\mathcal{H}_{t,i}|} \sum_{w \in \mathcal{H}_{t,i}} w, \quad \text{quad}|\mathcal{H}_{t,i}| \leq 5. \quad (4)$$

The client maintains a rolling history $\mathcal{H}_{t,i}$ of up to five models received from the server, including the current one, and returns their element-wise mean. Averaging produces a smooth, history-dependent response without local optimization. Because the client is not told which models are traps, every trap enters the same history and can affect several later responses.

FR4—Trajectory Imitation

$$d_s = w_{s,i}^{\text{send}} - w_{s-1,i}^{\text{send}}, \quad \hat{w}_{t,i} = w_{t,i}^{\text{send}} + \bar{d} + \eta, \quad (5)$$

where $\bar{d}$ is the mean of the latest five received-model differences and Gaussian η is normalized to 10% of $\|\bar{d}\|_2$. At cold start, the attacker uses a small Gaussian update. FR4 therefore attempts to imitate the direction and scale of recent global evolution, making it more plausible than memoryless noise. Nevertheless,

it uses every received model indiscriminately, so an undisclosed trap also contaminates its trajectory estimate.

Collusion, partial honest training, on-off participation, and defense-aware adaptation are outside the evaluated scope.

IV. SWT-CP METHODOLOGY

Detection primitives. For floating tensor k , a trap is generated as

$$\tau_k = w_{t,k} + 0.01 \max(\sigma(w_{t,k}), 10^{-4}) \xi_k, \quad \xi_k \sim \mathcal{N}(0, I). \quad (6)$$

The tensor-wise standard deviation makes the perturbation architecture- and scale-aware; the factor 0.01 keeps it small enough to remain trainable, while the 10^{-4} floor ensures that nearly constant tensors are still challenged. Responses are evaluated only against responses to the same trap. For reference norms R , SWT-CP computes median m_r , median absolute deviation d_r , and

$$z_{t,i}^{\text{norm}} = \frac{r_{t,i} - m_r}{\max(d_r, 0.05)}. \quad (7)$$

The magnitude signal fires when $|z_{t,i}^{\text{norm}}| > 3$. The absolute value detects both unusually weak and unusually strong responses. The threshold of three robust scales is deliberately conservative, and the 0.05 MAD floor prevents a nearly identical reference set from turning harmless numerical variation into an extreme score. For floating layer k , a scale-free profile is

$$p_{t,i,k} = \frac{\|\Delta_{t,i,k}\|_2}{\sum_j \|\Delta_{t,i,j}\|_2}. \quad (8)$$

Distance from the coordinate-wise median profile is standardized by median/MAD with floor 0.01; the profile signal also fires above three robust scales. Its smaller floor reflects the fact that profile coordinates are normalized to sum to one. Using the same three-scale rule for both signals avoids attack-specific tuning. Either signal can initiate candidate confirmation, but ordinary statistical penalties require both, so an honest client with only unusual update size or only unusual layer allocation is not punished. A response norm below 10^{-6} is treated separately as an exact-send-back tolerance rather than a learned statistical threshold.

Each active client occupies one of three states: unflagged (U), candidate (C), or suspicious (S). Let q_i denote its cumulative penalty, n_i its current-episode probe count, and f_i its joint-failure count. The detector then proceeds through the following phases.

A. Phase I: Warm-Up

For the first ten rounds, every client receives the ordinary global model. Ten rounds allow the initial global trajectory to stabilize and give history-based behavior a genuine model history before controlled probing begins. Nonzero population outliers are deliberately ignored because no controlled challenge has yet been issued. Only an exact or near-exact send-back is treated as evidence, adding five permanent points. Five is reserved for this strongest evidence so that three repeated send-backs reach the removal threshold of 15, while a single

Algorithm 1 Phase I: warm-up

Require: global state w_t , active clients V_t

```
1: for all  $i \in V_t$  do
2:   send  $w_t$  and receive  $w_{t,i}^{\text{ret}}$ 
3:   compute  $\Delta_{t,i}$ 
4:   if  $\|\Delta_{t,i}\|_2 < 10^{-6}$  then
5:      $q_i \leftarrow q_i + 5$ 
6:   end if
7: end for
8: remove  $i$  after three near-zero responses
9: aggregate all other ordinary responses with FedAvg
```

numerical accident cannot remove a client. This prevents natural early-training heterogeneity from populating the candidate set before the server has a trap-based reference. The procedure is summarized in Algorithm 1.

B. Phase II: Double Coverage

The server shuffles the 100-client population into ten base groups, giving a nominal group size of ten without making group membership observable to clients. In each round it probes two groups, preserving approximately 80% of clients for aggregation while completing one pass in five rounds. Two independently generated trap models, τ_A and τ_B , are used in randomized passes; every client therefore responds once to each trap over ten rounds. Two challenges prevent a single favorable or unfavorable trap response from deciding initial suspicion. Evidence is stored until both passes finish, and responses to τ_A and τ_B are calibrated separately so different challenges do not share a reference distribution.

Two joint magnitude/profile failures can immediately move a client to S : repeat agreement across both independent traps is required for direct escalation. A weaker anomaly moves it to C for fresh confirmation. Each joint coverage failure adds two points, an intermediate weight reflecting controlled but population-level evidence, while each near-zero response adds five. Every probed response is withheld from the global aggregation so deliberate trap updates cannot alter the production model. Algorithm 2 gives the complete sweep.

C. Phase III: Surveillance and Candidate Confirmation

Clients remaining in U are shuffled across ten surveillance groups. Retaining ten groups preserves the nominal ten-client comparison scale used during coverage. One group per round receives a shared surveillance trap; probing only one group preserves roughly 90% of ordinary contributions, and after ten rounds every group has been observed once. The server then evaluates the completed sweep and starts another with a new trap. A joint surveillance failure carries only one point because it is the broadest and most heterogeneous comparison. Any statistical anomaly places a client in C , where it is withheld from aggregation and receives a fresh group-specific trap in subsequent rounds.

Each candidate group is paired with up to three rotating anchors that have never been flagged. Three permits a median

Algorithm 2 Phase II: double coverage sweep

Require: active clients V , traps τ_A, τ_B

```
1: randomly partition  $V$  into ten base groups
2: for all  $\tau \in \{\tau_A, \tau_B\}$  do
3:   randomize a five-round pairing of the ten groups
4:   for all paired groups  $G$  do
5:     send  $\tau$  to every  $i \in G$ ; store  $\Delta_i$  and  $p_i$ 
6:     exclude  $G$  from the current aggregation
7:   end for
8:   compute norm/profile references using only responses
      to  $\tau$ 
9: end for
10: for all  $i \in V$  do
11:   add two points per joint failure and five per near-zero
      response
12:   if two joint failures then
13:     move  $i$  to  $S$ 
14:   else if either signal fires then
15:     move  $i$  to  $C$ 
16:   end if
17: end for
```

Algorithm 3 Phase III: surveillance and confirmation

```
1: probe the next of ten  $U$  groups with the sweep trap
2: if the surveillance sweep is complete then
3:   calibrate same-trap norm and profile statistics
4:   move anomalous clients  $U \rightarrow C$ 
5: end if
6: for all candidate groups  $C_g$  do
7:   rotate up to three never-flagged anchors  $A_g$ 
8:   generate fresh trap  $\tau_g$  and send it to  $C_g \cup A_g$ 
9:   compare candidates with the same-trap anchor refer-
      ence
10:  if candidate accumulates two joint confirmations then
11:    move candidate  $C \rightarrow S$ 
12:  else if candidate passes confirmation then
13:    return candidate  $C \rightarrow U$  and clear episode evidence
14:  end if
15: end for
16: withhold surveillance, candidate, and anchor trap responses
```

reference to tolerate one atypical anchor, while a global cap of ten anchors limits the number of honest updates withheld when several candidate or suspicious groups coexist. Candidates and anchors receive the same trap, making the anchor distribution a local reference for that challenge. Two joint candidate failures confirm S , again requiring repeated fresh-trap evidence rather than one outlier. A candidate that produces a normal confirmation is returned to U and its current magnitude episode is cleared so one false accusation does not bias later episodes. Off-duty anchors remain in ordinary surveillance and contribute normally. This rotating confirmation process appears in Algorithm 3.

Algorithm 4 Phase IV: weighted suspicion and decision

```
1: form approximately  $\max(1, \lfloor |S|/10 \rfloor)$  suspect groups
2: for all suspect groups  $S_g$  do
3:   attach rotating anchors  $A_g$  and generate fresh trap  $\tau_g$ 
4:   send  $\tau_g$  to  $S_g \cup A_g$  and withhold all responses
5:   for all  $i \in S_g$  do
6:      $n_i \leftarrow n_i + 1$ 
7:     if joint failure then
8:        $q_i \leftarrow q_i + 3$ ;  $f_i \leftarrow f_i + 1$ 
9:     end if
10:    if near-zero response then
11:      add five permanent points
12:    end if
13:    if  $n_i \geq 10$  and  $f_i/n_i \leq 0.10$  then
14:      rehabilitate  $i$  and clear magnitude-episode
15:    else if  $n_i \geq 10$  and  $q_i \geq 15$  then
16:      remove  $i$  from the federation
17:    end if
18:  end for
19: end for
20: aggregate only active, non-trapped, non-candidate re-
    sponses
```

D. Phase IV: Suspicion, Penalties, and Rehabilitation

Suspicious clients are divided into approximately one group per ten suspects and are challenged every round. A ten-client target keeps groups large enough for future resistance to simple response sharing without imposing a hard maximum group size as S shrinks. Each group receives its own fresh trap and rotating same-trap anchors. A joint suspicion failure adds three points because a same-trap, anchor-relative failure is stronger than population evidence; a near-zero response adds five. Statistical removal requires $q_i \geq 15$ and at least ten suspicion probes. The threshold therefore needs five strong anchor-relative failures, or three exact send-backs, rather than one extreme observation. Ten probes also provide a meaningful denominator for rehabilitation. After that minimum, a client with $f_i/n_i \leq 0.10$ —at most one joint failure in ten probes—is instead rehabilitated: it returns to U and its current magnitude penalty is cleared. This “mercy” rule prevents an initially false suspicion from keeping an otherwise consistent client under permanent intensive auditing. Near-zero penalties are permanent, so repeated exact send-backs cannot be erased through rehabilitation. The decision process is specified in Algorithm 4.

Lightweight implementation. For a model with P floating parameters and L tensors, processing one returned model requires one parameter pass, $O(P)$ arithmetic, and an L -dimensional profile. Medians operate on scalar norms and short profiles; the server neither recomputes gradients nor trains a detector. Client software performs ordinary local training on whichever model it receives and does not train a second model. Apart from model/trap tensors, server state consists mainly of

small per-client counters and bounded histories. Cost arises from probe training and temporary exclusion of trap responses, not an auxiliary learning pipeline.

V. EXPERIMENTAL DESIGN

MNIST [11] and CIFAR-10 [12] are trained with convolutional models using 100 fully participating clients, 100 rounds, three local epochs, batch size 32, and SGD with learning rate 0.01 and momentum 0.9. Data are IID or allocated by a Dirichlet distribution with concentration $\alpha = 0.5$. MNIST is evaluated with 30% and 40% free riders, while the available CIFAR-10 evaluation uses 40% free riders.

Experiments are repeated under multiple random seeds. Seed identifiers and execution counts are omitted from the tables. For this preliminary capability summary, each attack is represented by its best observed completed result across the evaluated seeds; configuration-level metrics then combine the four selected attack results. Consequently, the tables report demonstrated best-observed performance rather than mean or worst-case reliability.

A positive prediction means permanent removal. We report TP , FP , FN , precision, recall, and $F1$. Unresolved candidates are reported separately because they are quarantined but not permanently removed. Accuracy, runtime, memory, state transitions, anchor composition, and trap assignments are also logged.

VI. RESULTS

A. Best-Observed Multi-Seed Performance

Table I places both datasets, both partitions, and both attacker shares in one grouped view. It summarizes the best observed result for each attack across multiple seeds and then combines FR1–FR4 within each available configuration. Recall reaches 100% in every displayed configuration. MNIST precision ranges from 93.75% to 100%, whereas CIFAR-10 non-IID precision falls to 85.11%, showing that heterogeneous data primarily increases false removal rather than missed detection in the selected results. CIFAR-10 30% cells are marked as not evaluated because no corresponding experiments are available.

B. Utility and execution cost

Best-observed final accuracy is 98.89–98.93% on MNIST and 74.48–76.80% on CIFAR-10. Accuracy and detection remain distinct: a strong global model does not by itself establish that free riders were identified correctly.

Table II summarizes the recorded execution profile. Values first average repeated seeds within each attack and then weight FR1–FR4 equally. The 100-round duration is reconstructed from round logs, avoiding partial wall-clock values from resumed processes. Timings are descriptive rather than a controlled hardware benchmark: most configurations ran on CPU, while MNIST IID 40% used CUDA. Mean aggregated updates quantify a second practical cost, namely responses withheld by probing, quarantine, and removal.

Client simulations are serial, so these durations include local training rather than only detector overhead. Detection itself

TABLE I

BEST OBSERVED MULTI-SEED PERFORMANCE. EACH AVAILABLE SETTING COMBINES THE STRONGEST COMPLETED OBSERVATION FOR EACH OF FR1–FR4. A DASH DENOTES A SETTING THAT WAS NOT EVALUATED.

Dataset	Partition	30% free riders				40% free riders			
		Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy
MNIST	IID	99.17%	100%	99.59%	98.93%	100%	100%	100%	98.89%
MNIST	Non-IID	93.75%	100%	96.77%	98.90%	96.97%	100%	98.46%	98.89%
CIFAR-10	IID	–	–	–	–	100%	100%	100%	76.80%
CIFAR-10	Non-IID	–	–	–	–	85.11%	100%	91.95%	74.48%

TABLE II

RECORDED EXECUTION AND AGGREGATION PROFILE. PEAK RSS IS THE MEAN OF PER-EXECUTION PEAKS; AGGREGATED UPDATES ARE OUT OF 100 CLIENTS PER ROUND.

Dataset	Partition	Attackers	Device	Seconds/round	Hours/100 rounds	Peak RSS (GB)	Mean aggregated
MNIST	IID	30%	CPU	47.1	1.31	0.92	64.3
MNIST	IID	40%	CUDA	55.6	1.54	1.40	57.8
MNIST	Non-IID	30%	CPU	44.8	1.24	0.72	62.4
MNIST	Non-IID	40%	CPU	43.6	1.21	0.84	62.1
CIFAR-10	IID	40%	CPU	88.9	2.47	1.85	58.1
CIFAR-10	Non-IID	40%	CPU	79.5	2.21	1.90	53.6

adds differencing, profile construction, robust scalar statistics, counters, and trap generation, but no auxiliary-model training.

VII. DISCUSSION AND LIMITATIONS

The best-observed results show that SWT-CP can remove all four free-rider types under both IID and non-IID data. The principal remaining error is false removal under heterogeneous data, particularly on CIFAR-10, where honest update diversity weakens population and anchor-relative comparisons.

Because the present tables intentionally report the best observation across multiple seeds, they demonstrate capability but do not estimate average reliability or variance. A final evaluation should report all preregistered repetitions with uncertainty intervals and should separate partition, attacker, model, and trap randomness.

The study uses two image datasets, one non-IID concentration, full participation, and always-on non-colluding attackers. It does not test partial training, on–off behavior, defense-aware adaptation, dropout, or collusion. No external defense was rerun identically, so no comparative-superiority claim is made. Positive anchor qualification, repeated independent panels before candidate clearing, and paired within-client trap responses are priority extensions.

SWT-CP also requires attributable updates and temporarily spends honest computation on withheld responses. It is not directly compatible with ordinary secure aggregation [10]; private deployment would require securely computed audit statistics or a separate challenge protocol.

VIII. CONCLUSION

SWT-CP shows that secret server-generated traps and robust statistics can provide useful free-rider evidence without trusted client telemetry or a learned detector. Its best observed multi-seed results attain 100% recall across FR1–FR4 on MNIST and CIFAR-10 under both IID and non-IID partitions. Precision

remains high on MNIST and IID CIFAR-10 but declines under non-IID CIFAR-10, identifying heterogeneous honest updates as the next target for improving anchor qualification and calibration.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [2] K. Bonawitz, H. Eichner, W. Grieskamp *et al.*, “Towards federated learning at scale: System design,” in *Proceedings of Machine Learning and Systems*, vol. 1, 2019, pp. 374–388.
- [3] A. Hard, K. Rao, R. Mathews *et al.*, “Federated learning for mobile keyboard prediction,” *arXiv preprint arXiv:1811.03604*, 2018.
- [4] J. Lin, M. Du, and J. Liu, “Free-riders in federated learning: Attacks and defenses,” *arXiv preprint arXiv:1911.12560*, 2019.
- [5] Y. Fraboni, R. Vidal, and M. Lorenzi, “Free-rider attacks on model aggregation in federated learning,” in *Proc. AISTATS*, 2021, pp. 1846–1854.
- [6] Z. Zhu, J. Shu, X. Zou, and X. Jia, “Advanced free-rider attacks in federated learning,” in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021.
- [7] B. Wang, H. Li, X. Liu, and Y. Guo, “FRAD: Free-rider attacks detection mechanism for federated learning in AIoT,” *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024.
- [8] J. Wang, X. Chang, J. Mišić, V. Mišić, and Y. Wang, “PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack,” *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2023.
- [9] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, “FRIDA: Free-rider detection using privacy attacks,” *arXiv preprint arXiv:2410.05020*, 2024.
- [10] K. Bonawitz, V. Ivanov, B. Kreuter *et al.*, “Practical secure aggregation for privacy-preserving machine learning,” in *Proc. ACM CCS*, 2017, pp. 1175–1191.
- [11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [12] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.